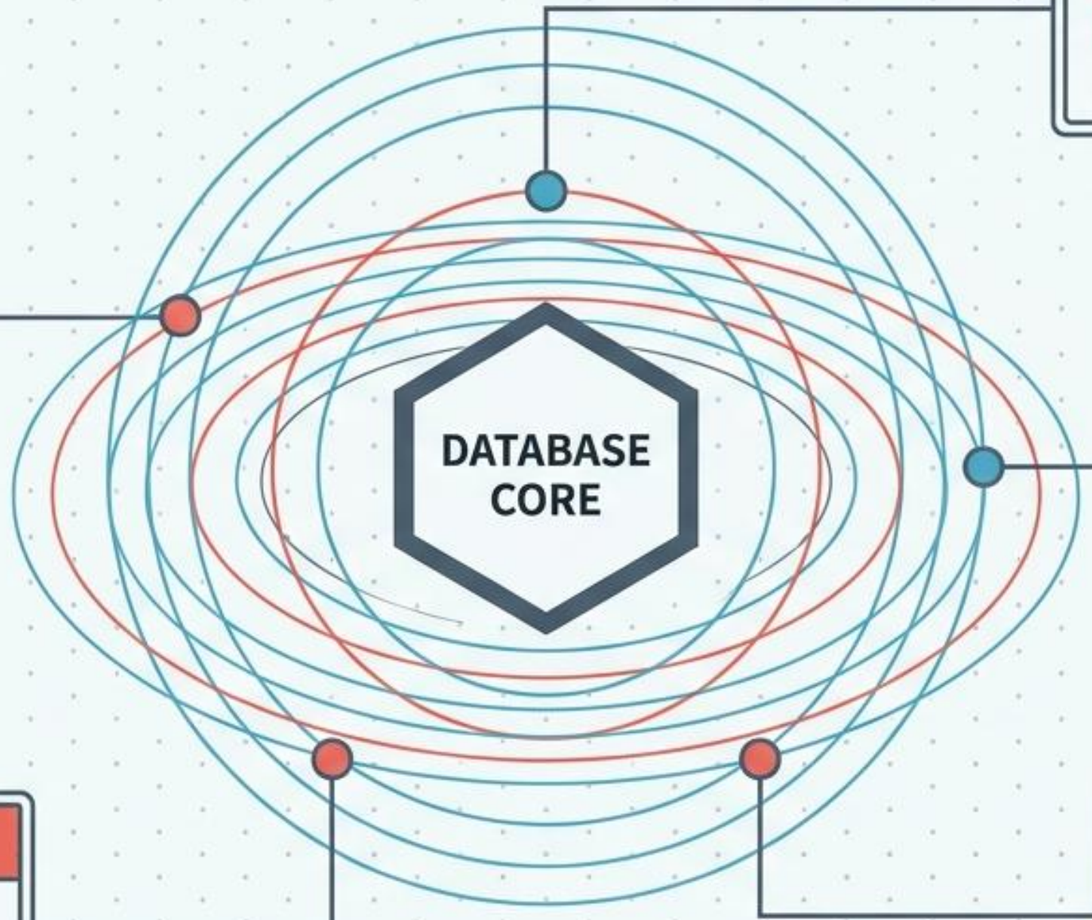
데이터베이스 코어를 움직이는 5가지 차원의 언어



DQL (데이터 검색)

SELECT
(데이터를 읽어오는 단일 핵심 통로)

DCL (데이터 제어)

GRANT, REVOKE
(접근 권한과 보안 통제)

DML (데이터 조작)

INSERT, UPDATE, DELETE
(데이터의 상태 변경)

TCL (트랜잭션 제어)

COMMIT, ROLLBACK, SAVEPOINT
(논리적 작업 단위의 영구 저장 및 취소)

DDL (데이터 정의)

CREATE, ALTER, DROP,
RENAME, TRUNCATE
(그릇과 구조 자체를 설계 및 파괴)

NotebookLM

데이터의 물리적 기반: 타입(Type)과 구조(Structure) 설계

원자재 툴박스

1. 문자열: CHAR(고정), VARCHAR2(가변, 최대 4KB)
2. 숫자: NUMBER(p, s) – 정밀도와 스케일 제어
3. 시간: DATE(기본 날짜), TIMESTAMP(밀리초 확장)
4. 대용량: BLOB(바이너리), CLOB(텍스트) – 최대 4GB

Table Lifecycle (구조 생명주기)

CREATE: 구조 생성
(서브쿼리 AS SELECT 복제 포함)

ALTER: 구조 변경
(ADD, MODIFY, DROP COLUMN)

DROP:
객체 완전 삭제

TRUNCATE:
공간만 남기고
데이터 초고속 초기화

NotebookLM

SELECT의 해부학: 데이터 필터링과 그룹핑의 파이프라인

1. FROM / JOIN
타겟 테이블 지정 및 결합 (원본 데이터 확보)

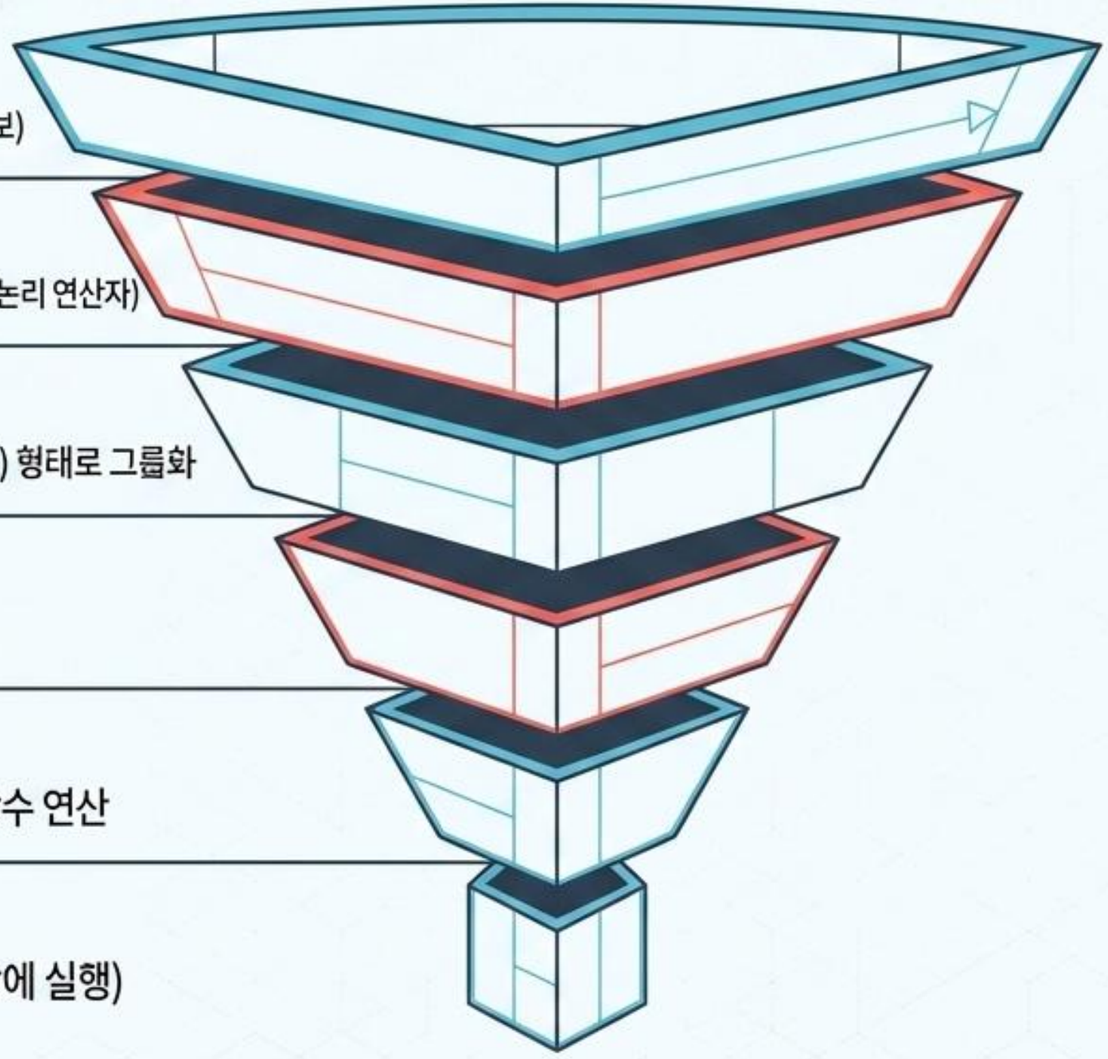
2. WHERE
조건에 맞지 않는 개별 행 필터링 (비교 및 논리 연산자)

3. GROUP BY
동일한 값을 가진 행들을 청크(Chunk) 형태로 그룹화

4. HAVING
그룹화된 결과에 대한 2차 필터링

5. SELECT
최종 추출할 칼럼 선택 및 그룹 함수 연산

6. ORDER BY
도출된 결과물 정렬 (가장 마지막에 실행)



```
SELECT ...
FROM ...
WHERE ...
GROUP BY ...
HAVING ...
ORDER BY ;
```

NotebookLM

데이터 성형 툴박스: 오라클 필수 내장 함수

문자/숫자 조작 (Shape)

SUBSTR(추출), INSTR(위치),
LPAD/RPAD(채우기), TRIM(공백제거),
ROUND/TRUNC(반올림/버림),
MOD(나머지)

날짜 연산 (Time)

SYSDATE(현재),
MONTHS_BETWEEN(개월 차이),
ADD_MONTHS(개월 더하기),
NEXT_DAY/LAST_DAY

형 변환 (Transform)

TO_CHAR(포맷팅 예: ‘YYYY.MM.DD’),
TO_DATE,
TO_NUMBER

논리 및 NULL 처리 (Logic)

NVL(대상, 기본값),
NVL2(대상, NotNull, Null),
DECODE(대상, 조건, 결과... 기본값),
CASE WHEN THEN ELSE END

NotebookLM

Light-mode Editorial Engineering Blueprint

조인의 기술: Oracle 구문 vs ANSI 표준 구문 완벽 비교

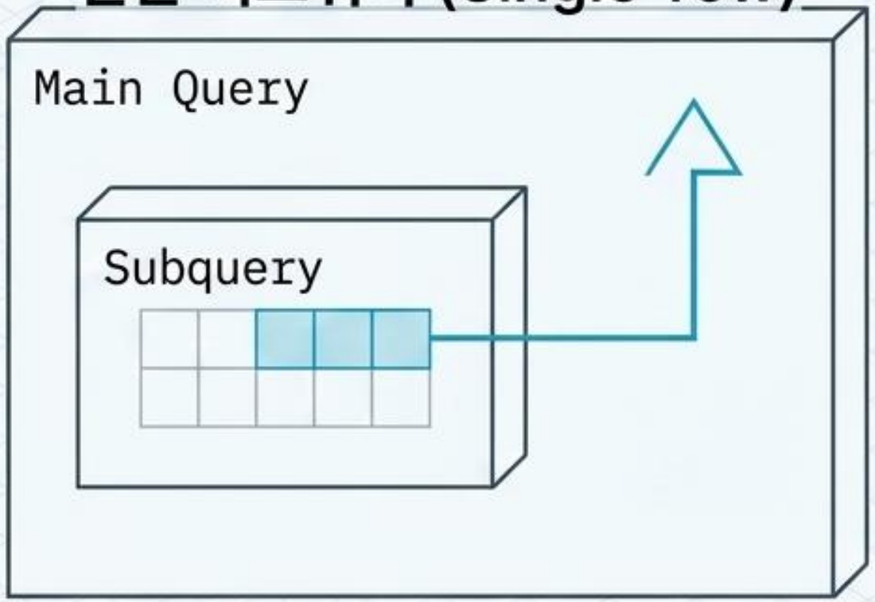
	Oracle Syntax	ANSI Standard Syntax
내부 조인 (INNER JOIN)	`WHERE e.deptno = d.deptno`	`INNER JOIN dept ON e.deptno = d.deptno`
외부 조인 (OUTER JOIN)	`WHERE e.deptno = d.deptno(+)`	`LEFT/RIGHT OUTER JOIN ... ON`
크로스 조인 (CROSS JOIN)	(조건 없는 콤마 연결)	`CROSS JOIN`
완전 외부 조인 (FULL OUTER)	(지원 안 함)	`FULL OUTER JOIN`

Highlight Note: ANSI 표준은 조인 조건(ON)과 필터링 조건(WHERE)을 명확히 분리하여 가독성이 높습니다.

NotebookLM

데이터 네비게이션: 서브쿼리(Subquery)의 중첩 로직

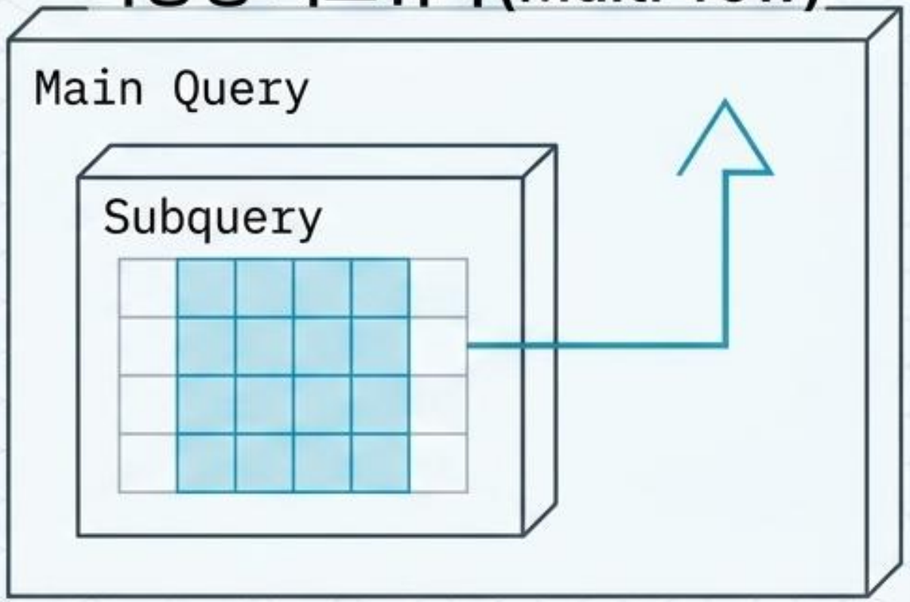
일반 서브쿼리 (Single-row)



단일 값을 메인 쿼리에 전달.

연산자: =, >, <

다중행 서브쿼리 (Multi-row)



여러 행(List)으로 반환.

연산자:

- IN: 하나라도 일치하면 참
- ANY / SOME: 하나 이상과 비교하여 만족 시 참
- ALL: 모든 값과 비교하여 만족 시 참
- EXISTS: 결과 존재 여부만 판별

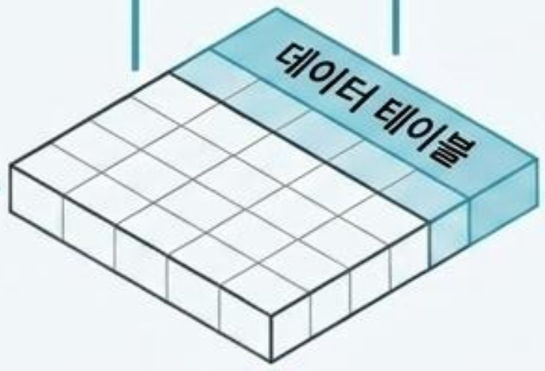
NotebookLM

Light-mode Editorial Engineering Blueprint

철벽의 무결성: 5대 제약조건(Constraints) 아키텍처

PRIMARY KEY (기본키)
행 고유 식별자
(UNIQUE + NOT NULL).
테이블당 1개.

FOREIGN KEY (외래키)
타 테이블 기본키 참조.
참조 무결성 유지.



UNIQUE (유일키)
중복 값 허용 불가
(NULL은 가능).

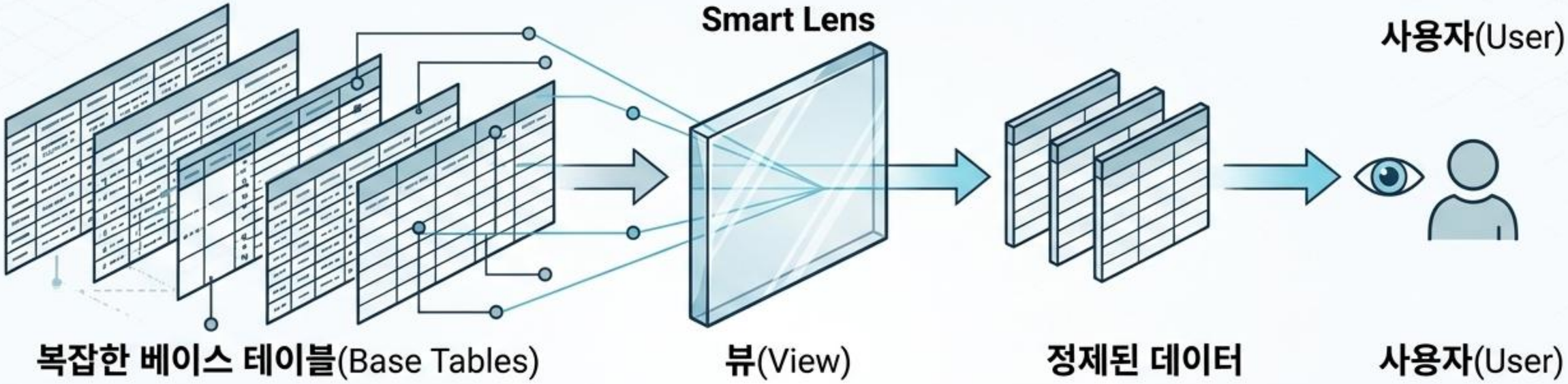
NOT NULL
NULL 값 입력 전면 차단
(필수값).

CHECK
설정된 범위나 조건에
맞는 데이터만 허용.

ALTER TABLE을 통해 비활성화(DISABLE) 및 재활성화(ENABLE) 유연 관리 가능.

NotebookLM

데이터 가상화: 뷰(View)와 인라인 뷰(Inline View)



뷰(View): 실제 데이터를 저장하지 않고 쿼리문만 품고 있는 가상의 테이블

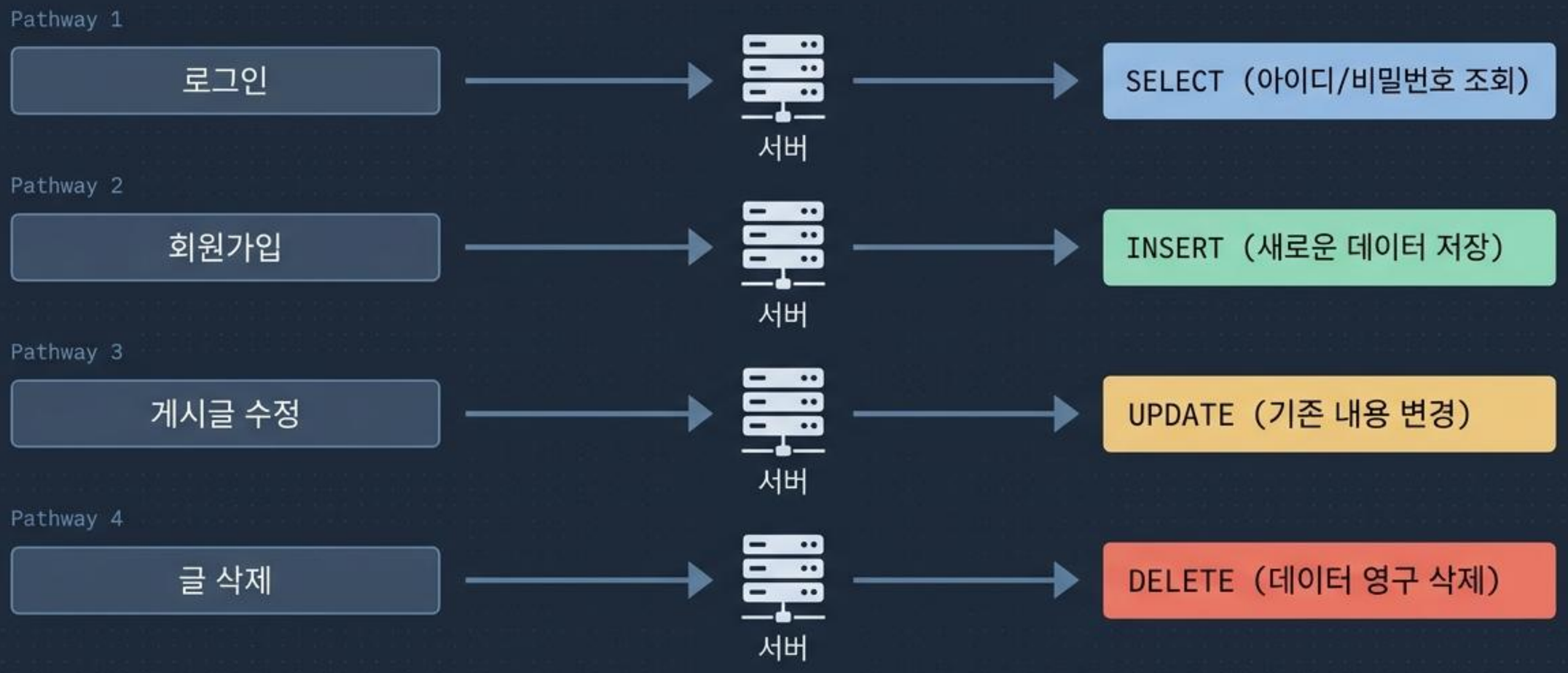
보안성 (Security)
민감한 칼럼 숨김. WITH READ ONLY 옵션으로 DML 전면 차단.

편의성 (Simplicity)
복잡한 JOIN/GROUP 연산을 미리 생성하여 단일 테이블처럼 조회.

인라인 뷰 (Inline View) : FROM 절 내부에 즉석에서 작성되는 일회성 서브쿼리.
데이터를 정렬(ORDER BY) 후 로우 순서(ROWNUM)를 매겨 Top-N 추출 시 필수적.

NotebookLM

모든 사용자의 행동은 네 가지 데이터베이스 명령으로 번역됩니다.



이 4가지 작업(CRUD)이 전체 웹 개발 데이터 처리의 80%를 차지합니다.

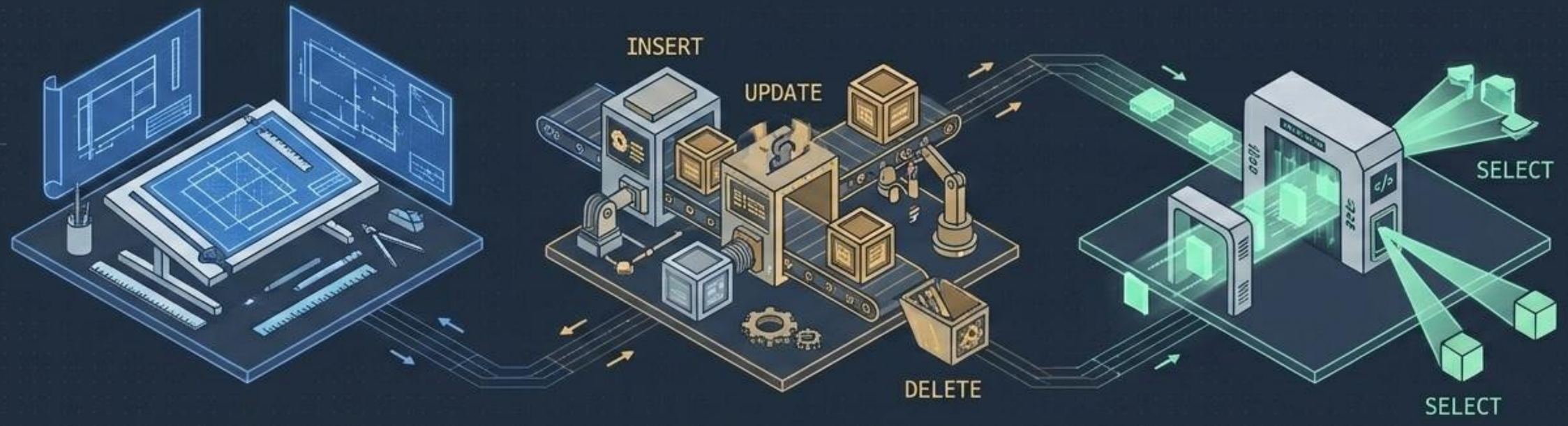
NotebookLM

오라클 데이터베이스는 세 개의 핵심 구역으로 나뉜 거대한 공장입니다.

DDL
구조 설계 및 테이블 생성

DML
CRUD 로직 실행
(INSERT, UPDATE, DELETE)

DQL
필요한 정보만 빠르게 추출
(SELECT)

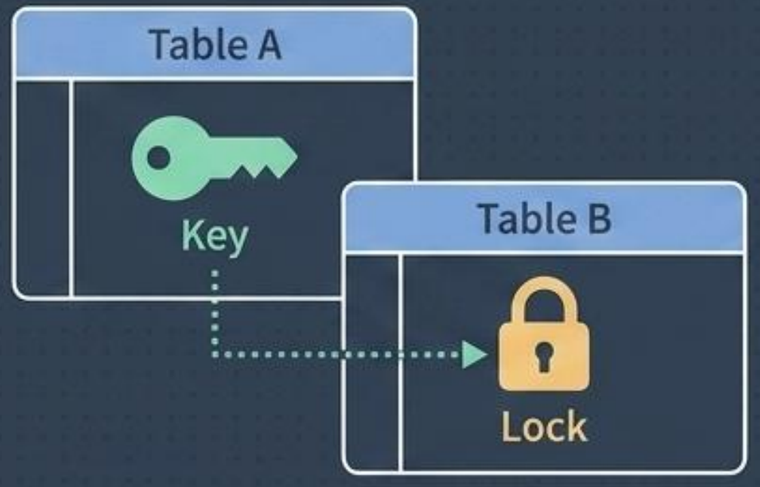


Oracle Database: 데이터를 단순히 저장하는 것을 넘어, 철저히 관리하고 가장 빠르게 찾아내는 최적의 시스템.

NotebookLM

견고한 데이터베이스는 명확한 타입과 강력한 제약조건이라는 기초 위에 세워집니다.

제약조건: 무결성의 자물쇠



- PRIMARY KEY (고유+필수)
- FOREIGN KEY (다른 테이블 참조)
- NOT NULL (필수 입력)
- UNIQUE (중복 금지)
- CHECK (값 제한)

실무 필수 데이터 타입 3선

VARCHAR2	문자열
NUMBER	숫자
DATE	날짜

실무에서는 거의 이 3가지만으로 구조가 완성됩니다.

NotebookLM

데이터가 추출되는 과정은 정해진 컨베이어 벨트 순서를 엄격히 따릅니다.

1. **FROM** (어디서 가져올 것인가?)
2. **WHERE** (어떤 조건으로 거를 것인가?)
3. **GROUP BY** (어떻게 묶을 것인가?)
4. **HAVING** (묶은 그룹 중 무엇을 남길 것인가?)
5. **SELECT** (어떤 컬럼만 보여줄 것인가?)
6. **ORDER BY** (어떤 순서로 정렬할 것인가?)

코드 작성 순서(SELECT -> FROM...)와 실제 실행 순서는 다릅니다.
실행 흐름을 이해해야 성능이 보입니다.

NotebookLM

WHERE 절은 방대한 데이터의 바다에서 필요한 것만 걸러내는 정밀한 필터입니다.

수백만 건의 데이터가 단 몇 건의 핵심 정보로 압축됩니다.

NotebookLM

NULL은 '값이 없음'이 아니라 '알 수 없음'이므로 일반 기호로는 포착할 수 없습니다.

초보자의 함정

```
WHERE comm = NULL
```

NULL은 비교가 불가능하므로,
이 쿼리는 아무것도 찾지 못합니다.

실무자의 정답

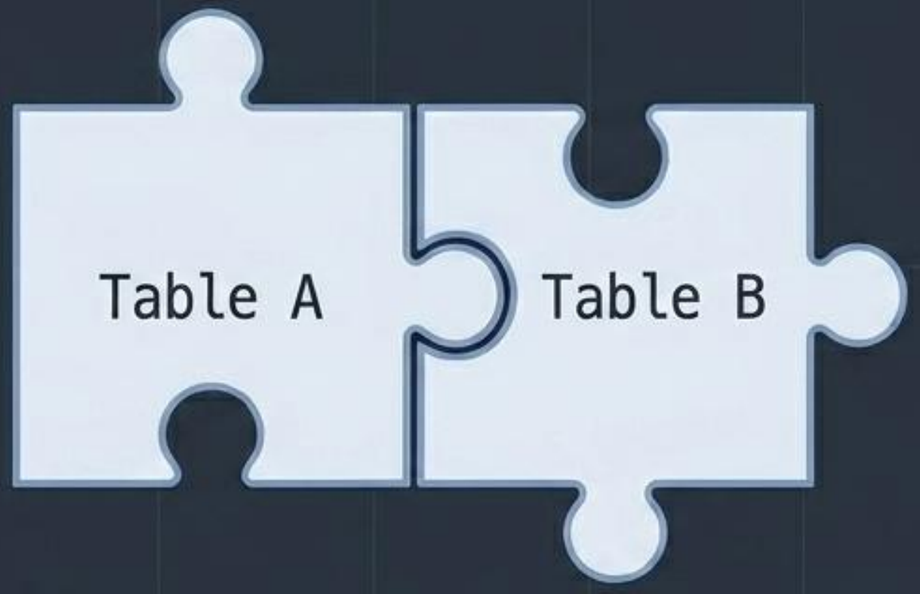
```
WHERE comm IS NULL
NVL(comm, 0)
```

존재 여부를 묻는 IS NULL을 사용하거나,
NVL을 통해 안전한 기본값(0)으로
변환 후 계산해야 합니다.

NotebookLM

쪼개진 데이터 조각들을 결합하여
하나의 완전한 화면을 구성하는 접착제, JOIN.

데이터 조립



INNER JOIN (교집합)

OUTER JOIN (기준 테이블 전체 유지)

Legacy vs ANSI

과거 방식 (Legacy)

```
FROM A, B
WHERE A.id = B.id
```

조건과 조인이 섞여 가독성 저하

실무 표준 (ANSI)

```
FROM A JOIN B
ON A.id = B.id
```

조인 의도가 명확히 분리됨, 유지보수 최적화

NotebookLM

원시 데이터는 그룹화와 가공 함수를 거쳐 비즈니스를 위한 통계 지표로 변환됩니다.

```
SELECT deptno, COUNT(*)
FROM   emp
GROUP BY deptno;
```

COUNT

SUM / AVG

SUBSTR / TO_CHAR

IF / TO_CHAR

웹 서비스의 관리자 페이지(Admin)는 이 문법들의 조합으로 완성됩니다.

NotebookLM

복잡한 조건은 쿼리리 속의 쿼리와 가상 테이블로 단순화합니다.

서브쿼리 (Subquery)

메인 쿼리가 실행되기 전, 내부에서 필요한 값을 먼저 도출해내는 단일행/다중행 보조 쿼리.

뷰 (View)

복잡한 쿼리를 하나의 가상 테이블(CREATE VIEW)로 저장하여 보안과 재사용성을 극대화.

실무 대원칙: JOIN과 서브쿼리는 상황에 따라 성능 차이가 발생하므로 둘 다 능숙해야 합니다.

NotebookLM

단 한 줄의 실수로 전체 데이터를 날리는 것을 막기 위한 최종 안전 장치.

Transaction Controls

COMMIT
모든 변경사항을 영구 저장

ROLLBACK
직전 상태로 즉시 취소 및 복구

실무 치명적 실수 체크리스트

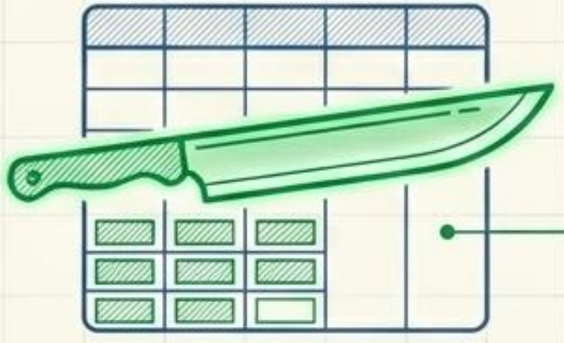
- [ ] WHERE 조건절이 빠진 UPDATE / DELETE (전체 데이터 날아감)
- [ ] JOIN 시 연결 조건 누락 (Cartesian Product 발생, DB 터짐)
- [ ] 습관적인 SELECT * 남용 (서버 메모리 과부하)

NotebookLM

DDL (데이터 정의 언어): 데이터베이스 빌더

CRUCIAL DISTINCTION: DROP vs. TRUNCATE

DROP:
객체 완전 삭제.
GC(Garbage Collector)에
의해 null 처리됨.



TRUNCATE:
데이터만 잘라내기(초기화).
테이블 구조는 유지되나
ROLLBACK 불가.

NotebookLM

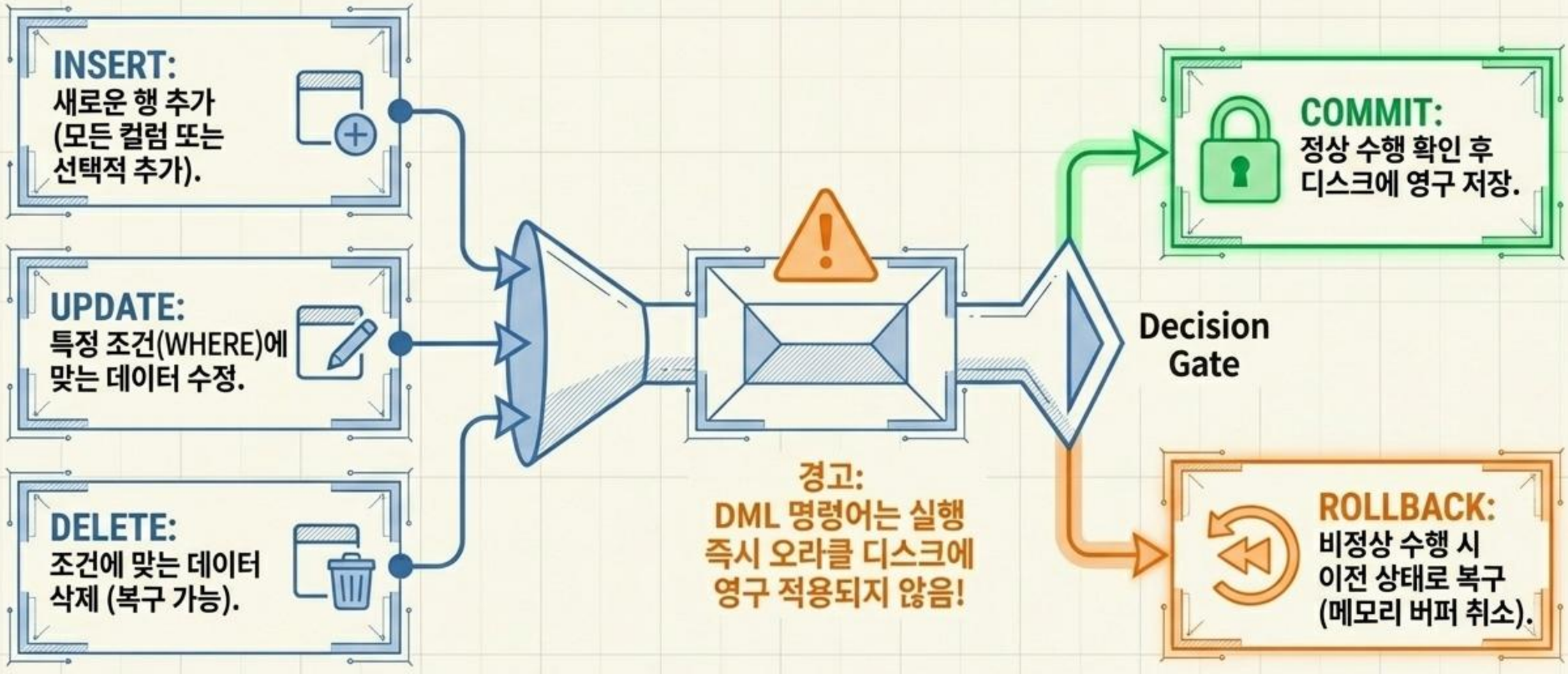
제약조건 (Constraint) 진단 매트릭스

비정형 데이터를 정형화하고 이상 현상을 방지하는 필수 규칙.

제약조건	NULL 허용?	중복 허용?	핵심 목적
PRIMARY KEY	🟢 불가 (X)	🟠 불가 (X)	테이블당 1개. 식별자. (NOT NULL + UNIQUE).
FOREIGN KEY	🟢 허용 (O)	🟢 허용 (O)	참조 무결성 유지 (ON DELETE CASCADE 활용).
UNIQUE	🟢 허용 (O)	🟠 불가 (X)	대체키/ 후보키. 고유값 보장 (예: 이메일).
NOT NULL	🟠 불가 (X)	🟢 허용 (O)	필수 입력값 강제.
CHECK	N/A	N/A	지정된 데이터 값만 허용 (예: 성별, 부서명).

NotebookLM

DML (데이터 조작 언어)과 트랜잭션 제어 파이프라인



NotebookLM

핵심 데이터베이스 객체: 보안, 속도, 자동화

1. VIEW (가상 테이블 / 보안성)

설명: 하나 이상의 테이블을 합친 읽기 전용(WITH READ ONLY) 추상화 계층.

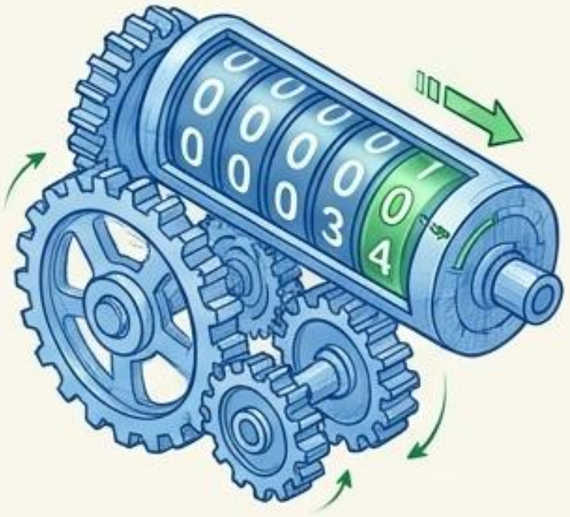
목적: 복잡한 SQL 쿼리 재사용 및 민감한 데이터 숨김 (디스크에 저장되지 않음).

2. INDEX (속도 최적화)

설명: 데이터 검색 속도(SELECT)를 비약적으로 높이는 목차.

주의: 잦은 INSERT/UPDATE/DELETE 발생 시 성능 저하 (주기적인 REBUILD 필요). PRIMARY KEY는 자동 인덱스 생성.

3. SEQUENCE (자동화된 카운터)



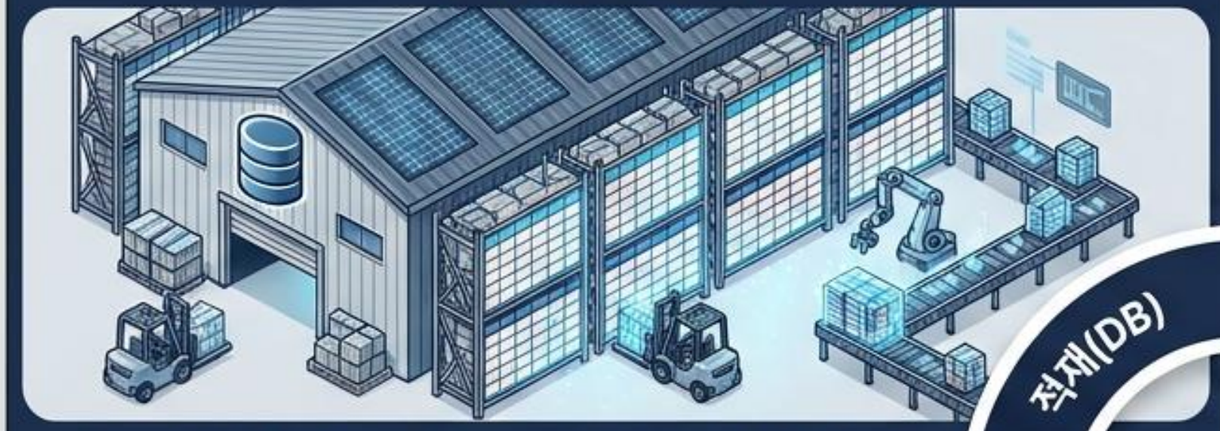
설명: 중복 없는 고유 숫자를 순차적으로 생성 (NEXTVAL).

목적: 게시판 번호, 주문 번호 등 자동 증가(Auto-increment) 구현.

NotebookLM

데이터 물류 센터: 4대 핵심 컴포넌트의 역할 분담

[창고] Oracle DB



데이터가 엑셀 표처럼 안전하게 적재되어 있는 거대한 중앙 물류 창고

[배송 트럭] JDBC

창고(DB)와 외부 애플리케이션(Java)을 연결하는 전용 운송 인프라 네트워크

적재(DB)

배차(JDBC)

포장(VO)

지시(DAO)

[직원] DAO (Data Access Object)

창고에서 물건을 꺼내 포장하고 지시(SQL 실행)를 내리는 현장 총괄 관리자

[박스] VO (Value Object)

창고에서 꺼낸 단 하나의 데이터 행(ROW)을 안전하게 담아 나르는 규격화된 택배 상자

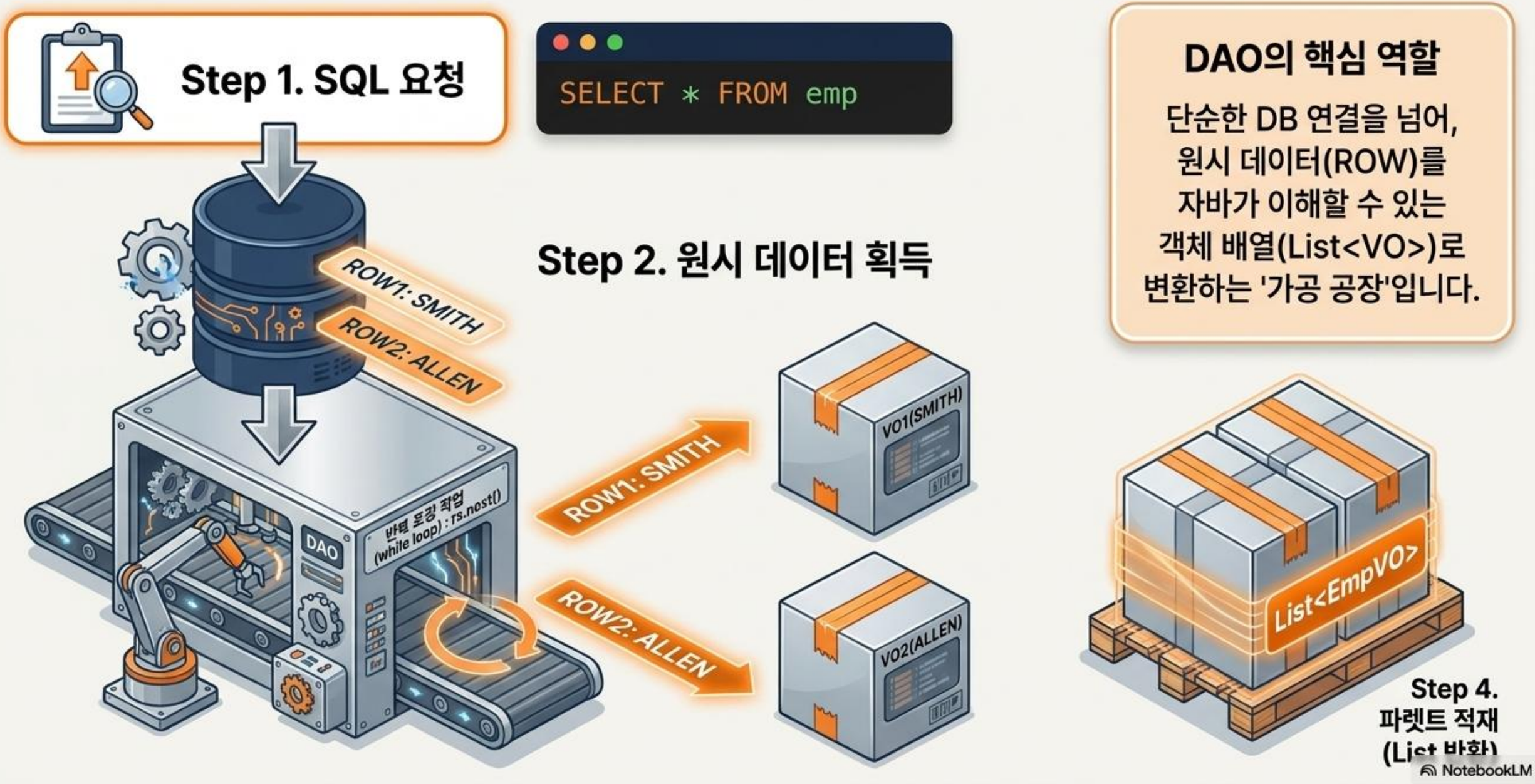
NotebookLM

데이터 패킹 공식: DB의 한 줄(ROW) = VO 객체 하나

⚠ 결코 한 VO에 여러 ROW가 담기지 않습니다. 한 명의 정보는 반드시 하나의 독립된 박스에 포장됩니다.

NotebookLM

DAO 공장의 컨베이어 시스템: 다중 데이터의 처리



DAO의 핵심 역할

단순한 DB 연결을 넘어,
원시 데이터(ROW)를
자바가 이해할 수 있는
객체 배열(List<VO>)로
변환하는 '가공 공장'입니다.

JDBC 6단계 마스터 노선도 (The 6-Step Pipeline)

💡 이 흐름은 모든 JDBC 작업의 불변하는 절대 공식입니다.

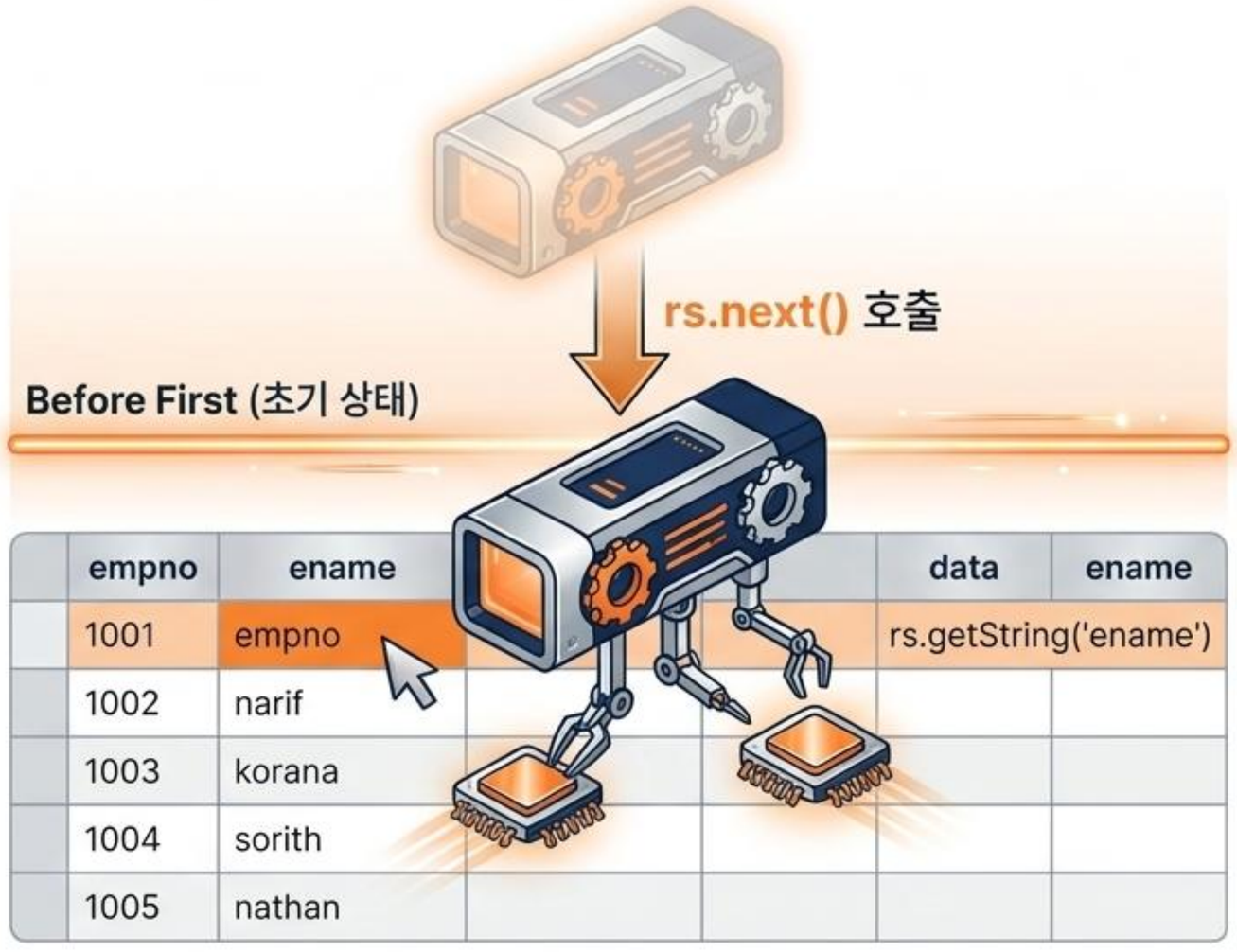
NotebookLM

SQL 실행 엔진 비교: 왜 실무는 무조건 PreparedStatement인가?

	Statement	PreparedStatement
SQL 처리 방식	실행 시마다 매번 컴파일	**DB에서 사전 컴파일** (캐싱)
속도 & 성능	낮음	**높음** (반복 실행 시 압도적)
보안	○ **취약** (문자열 결합으로 인한 SQL Injection 위험)	○ **절대 안전** (값과 쿼리의 구조 분리)
데이터 바인딩 방식	' + name + ' (지저분함)	**?** (깔끔한 파라미터 매핑)

NotebookLM

결과 집합 스캐닝: ResultSet과 Cursor 메커니즘



1. **executeQuery()** 실행 시 테이블 형태의 ResultSet 반환.

2. **while(rs.next())** 루프가 돌 때마다 커서가 한 줄씩 전진.

3. 더 이상 읽을 데이터가 없으면 false를 반환하며 루프 종료.

NotebookLM

실전 구축 I: SELECT와 객체 리스트 반환

Terminal

```
String sql = 'SELECT * FROM emp ORDER BY ename';

ResultSet rs = pstmt.executeQuery();
while(rs.next()) {
    EmpVO vo = new EmpVO();
    vo.setEname(rs.getString('ename'));
    list.add(vo);
}
```

[지시서 작성]

[조회 전용 실행 메서드]

[빈 상자 준비]

[박스에 데이터 채우기]

[파렛트에 적재]

💡 날짜 형식이 필요할 경우 자바에서 변환하기보다 SQL의 to_char(hiredate,'YYYY.MM.DD')를 활용하는 것이 실무적입니다.

NotebookLM

안전 프로토콜: 자원 해제의 절대 철칙 (역순 법칙)

Terminal

```
finally {
    try {
        if(rs != null) rs.close();
        if(pstmt != null) pstmt.close();
        if(conn != null) conn.close();
    } catch(SQLException e) {}
}
```

Warning: Null Check(!= null)는 필수입니다.
자원을 닫지 않으면 메모리 누수(Leak)가 발생하여
결국 전체 시스템이 다운됩니다.

NotebookLM